

Projet : AKSUS_Immob

Support Final – AKSUS_Immob

Codage par section – Développement & Déploiement simplifié

AFRIK DATA AND TECHNOLOGY – dataafrik@gmail.com – +229 01 97 79 56 11

Comprendre

PROJET AKSUS_Immob.

Un projet de conception pour le Bénin, en intégrant les spécificités du cadre juridique local (Code foncier et domanial, ANDF, loi sur les transactions électroniques) et la plateforme e-Notaire qui sécurise les actes authentiques pour les investisseurs et la diaspora. L'application conserve son ouverture à tous (particuliers, agents agréés, notaires, promoteurs) et son modèle freemium/premium.

Contexte juridique et objectifs au Bénin

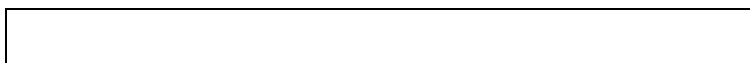
L'application s'appuie sur :

- Loi n° 2013-01 du 14 août 2013 portant code foncier et domanial, qui crée l'Agence Nationale du Domaine et du Foncier (ANDF) et instaure le titre foncier unique.
- Décret n° 2015-012 fixant les modalités de délivrance du titre foncier par l'ANDF.
- Loi n° 2017-20 du 20 avril 2018 relative aux transactions électroniques et à la signature électronique, qui reconnaît la valeur juridique des actes dématérialisés.
- Plateforme e-Notaire du Bénin, opérée par le Conseil Supérieur du Notariat, qui permet la rédaction, la signature et l'archivage des actes notariés en ligne, avec force probante.

Objectifs :

- Permettre à tout Béninois (y compris la diaspora) de publier, rechercher et vendre un bien immobilier en toute sécurité.
- Vérifier automatiquement la validité du titre foncier (TF) via une interconnexion avec l'ANDF.
- Générer et signer des avant-contrats (promesses de vente) avec valeur juridique, puis basculer vers la plateforme e-Notaire pour l'acte authentique final.
- Offrir une traçabilité totale et une protection contre la fraude (double vente, faux titres).
- Proposer un parcours gratuit pour les consultations de base et des abonnements payants pour les transactions engageantes.

1. Architecture générale (contexte béninois)

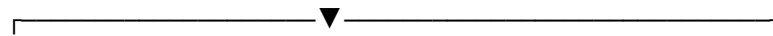


| UTILISATEURS (web, mobile, USSD) |

| Bénin + diaspora, particuliers, agents, notaires |



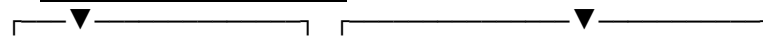
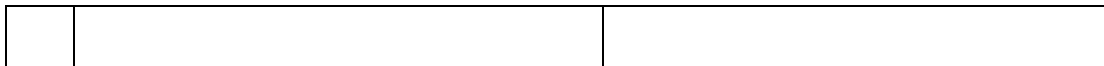
|



| API Gateway (sécurité, throttling) |

| Authentification : OAuth2 / FranceConnect ? |

| mais plutôt eID Bénin (ANIP ?) ou création compte |



| SERVICE USERS | SERVICE MÉTIER IMMOBILIER |

| (profil, abon.) | Annonces, transactions |

| _____ | Génération documents |

| | Signature simple/avancée |

| | _____ |



| | _____ |

| | CONNECTEUR ANDF (API) |

| | Vérification titre foncier |

| | Consultation cadastre |

| | _____ |



| | _____ |

| | CONNECTEUR E-NOTAIRE |

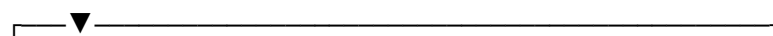
| | Création dossier notarié |

| | Signature qualifiée |

| | Archivage légal |

| | _____ |

|



	MESSAGE BROKER + STOCKAGE	
	PostgreSQL, MongoDB, S3, Elasticsearch, Vault	

Points d'intégration externes :

- API ANDF : consultation du fichier national des titres fonciers (vérification existence TF, identité du propriétaire, charges, hypothèques) – basée sur un protocole REST sécurisé avec certificat.
- API e-Notaire : authentification notaire, création de dossier de vente, envoi du projet d'acte, récupération du statut et de la preuve de signature qualifiée.

1. ARCHITECTURE DÉPLOYABLE (DOCKER COMPOSE)

Fichier docker-compose.yml racine – environnement complet en une commande.

```
```yaml
```

Version : '3.8'

Services :

Db :

Image : postgres:15-alpine

Environment :

POSTGRES\_DB : aksus

POSTGRES\_USER : aksus

POSTGRES\_PASSWORD : strongpassword

Volumes :

- Pgdata:/var/lib/postgresql/data

Ports :

- « 5432 :5432 »

Redis :

Image : redis:7-alpine

Ports :

- « 6379 :6379 »

Api :

Build : ./backend

Ports :

- « 3000 :3000 »

Depends\_on :

- db
- redis

Environment :

DATABASE\_URL : postgresql://aksus :strongpassword@db :5432/aksus

REDIS\_URL : redis://redis :6379

JWT\_SECRET : change-me-in-production

UPLOAD\_DIR : /app/uploads

Frontend :

Build : ./frontend

Ports :

- « 80 :80 »

Depends\_on :

- Api

Volumes :

Pgdata :

...

## 2. BASE DE DONNÉES – MODÈLE RELATIONNEL (SQL)

Script d'initialisation backend/prisma/schema.prisma (utilise Prisma ORM pour simplicité).

```prisma

Model User {

Id String @id @default(uuid())

Email String @unique

Phone String ?

passwordHash String

role Role @default(BUYER)

```

kycStatus    KycStatus @default(PENDING)
preferredLang String  @default(« fr »)
createdAt    DateTime @default(now())
updatedAt    DateTime @updatedAt
properties   Property[] @relation(« Owner »)
transactionsAsBuyer Transaction[] @relation(« Buyer »)
transactionsAsSeller Transaction[] @relation(« Seller »)
}

```

Model Property {

```

Id          String @id @default(uuid())
ownerId     String
owner       User   @relation(« Owner », fields : [ownerId], references : [id])
type        PropertyType
title       String
description String
price       Decimal
currency    String @default(« XOF »)
surface     Float ?
rooms       Int ?
landTitleNum String ?
verificationStatus VerificationStatus @default(PENDING)
status      PropertyStatus @default(DRAFT)
createdAt   DateTime @default(now())
updatedAt   DateTime @updatedAt
transactions Transaction[]
}

```

Model Transaction {

```

Id          String @id @default(uuid())
buyerId     String

```

```

sellerId    String
propertyId  String
buyer       User   @relation(« Buyer », fields : [buyerId], references : [id])
seller      User   @relation(« Seller », fields : [sellerId], references : [id])
property    Property @relation(fields : [propertyId], references : [id])
status      TransactionStatus @default(INITIATED)
offerAmount Decimal ?
depositAmount Decimal ?
preContractHash String ?
signedAt    DateTime ?
completedAt DateTime ?
payments    Payment[]
documents   Document[]
createdAt   DateTime @default(now())
updatedAt   DateTime @updatedAt
}

```

```

Model Payment {
  Id      String @id @default(uuid())
  transactionId String
  transaction Transaction @relation(fields : [transactionId], references : [id])
  amount   Decimal
  currency String @default(« XOF »)
  method   PaymentMethod
  externalRef String ?
  status    PaymentStatus @default(PENDING)
  createdAt DateTime @default(now())
}

```

```

Model Document {
  Id      String @id @default(uuid())

```

```
transactionId String

transaction Transaction @relation(fields : [transactionId], references : [id])

type DocumentType

url String

hash String

uploadedAt DateTime @default(now())

}
```

```
Enum Role {

    BUYER

    SELLER

    AGENT

    NOTARY

    ADMIN

}
```

```
Enum KycStatus {

    PENDING

    VERIFIED

    REJECTED

}
```

```
Enum PropertyType {

    HOUSE

    APARTMENT

    LAND

    COMMERCIAL

    PARKING

}
```

```
Enum VerificationStatus {
```

```
PENDING  
VERIFIED  
REJECTED  
}
```

```
Enum PropertyStatus {  
    DRAFT  
    PUBLISHED  
    SUSPENDED  
    SOLD  
}
```

```
Enum TransactionStatus {  
    INITIATED  
    OFFER_MADE  
    OFFER_ACCEPTED  
    PRE_CONTRACT_SIGNED  
    DEPOSIT_PAID  
    COMPLETED  
    CANCELLED  
}
```

```
Enum PaymentMethod {  
    MTN_MOMO  
    MOOV_MONEY  
    ORANGE_MONEY  
    WAVE  
    BANK_TRANSFER  
    CARD  
}
```

```
Enum PaymentStatus {  
    PENDING  
    SUCCESS  
    FAILED  
}
```

```
Enum DocumentType {  
    PRE_CONTRACT  
    SIGNED_PRE_CONTRACT  
    DEED_OF_SALE  
    PAYMENT_PROOF  
    ID_CARD  
    PROPERTY_TITLE  
}  
...
```

3. AUTHENTIFICATION & SÉCURITÉ (JWT + KYC)

Backend/src/auth/auth.controller.ts

```
``typescript
```

```
Import { Controller, Post, Body, UnauthorizedException } from '@nestjs/common' ;
```

```
Import { AuthService } from './auth.service' ;
```

```
Import { LoginDto, RegisterDto } from './dto' ;
```

```
@Controller('auth')
```

```
Export class AuthController {
```

```
    Constructor(private authService : AuthService) {}
```

```
    @Post('register')
```

```
    Async register(@Body() dto : RegisterDto) {
```

```
Return this.authService.register(dto) ;  
}
```

```
@Post('login')  
Async login(@Body() dto : LoginDto) {  
  Const user = await this.authService.validateUser(dto.email, dto.password) ;  
  If ( !user) throw new UnauthorizedException('Identifiants invalides') ;  
  Return this.authService.login(user) ;  
}
```

```
@Post('kyc')  
Async submitKyc(@Body() kycData : any, @Req() req) {  
  // Vérification pièce d'identité, selfie, justificatif  
  // Mise à jour kycStatus = VERIFIED  
}  
}  
...
```

JWT signé avec secret, expiration 7 jours, rotation refresh token.

4. MODULE ANNONCES (CRUD + RECHERCHE ELASTICSEARCH)

Backend/src/properties/properties.controller.ts

```
``typescript  
@Controller('properties')  
Export class PropertiesController {  
  Constructor(private propertiesService : PropertiesService) {}  
  
  @Post()  
  @UseGuards(JwtAuthGuard, RolesGuard) // Vendeur ou Agent
```

```

Async create(@Body() dto : CreatePropertyDto, @Req() req) {
    Return this.propertiesService.create(req.user.id, dto) ;
}

```

```

@Get()
Async search(@Query() query : SearchQueryDto) {
    Return this.propertiesService.search(query) ;
}

```

```

@Get('/:id')
Async findOne(@Param('id') id : string) {
    Return this.propertiesService.findOne(id) ;
}

```

```

@Patch('/:id/verify')
@UseGuards(RolesGuard) // Admin ou Notaire
Async verify(@Param('id') id : string, @Body() body : { status : VerificationStatus }) {
    Return this.propertiesService.updateVerification(id, body.status) ;
}
}
...

```

Le service utilise Elasticsearch pour la recherche multilingue (indexation du titre et description en fr, en, pt).

5. VÉRIFICATION DES BIENS (SIMULATION API CADASTRALE)

```

```typescript
// backend/src/properties/verification.service.ts
Export class VerificationService {
 Async checkLandTitle(landTitleNum : string, country : string) : Promise<VerificationResult> {

```

```

// Appel à un service tiers (cadastre béninois) ou simulation
// En environnement de développement, réponse mockée
Return {
 Valid : true,
 ownerMatch : true,
 details : 'Titre foncier authentique'
};
}
}
...

```

## 6. WORKFLOW JURIDIQUE & TRANSACTIONS

Machine à états dans backend/src/transactions/transaction.service.ts

```typescript

```

Export class TransactionService {
  Async makeOffer(transactionId : string, buyerId : string, amount : number) {
    // transition : INITIATED -> OFFER_MADE
    // Notifier vendeur
  }

  Async acceptOffer(transactionId : string, sellerId : string) {
    // transition : OFFER_MADE -> OFFER_ACCEPTED
  }

  Async signPreContract(transactionId : string, userId : string, signatureData : string) {
    // Vérifier signature électronique via partenaire (ex. Lex Persona API)
    // Stocker hash du document signé
    // transition : OFFER_ACCEPTED -> PRE_CONTRACT_SIGNED
  }
}

```

```

Async confirmDeposit(transactionId : string, paymentId : string) {
  // Après confirmation du paiement séquestre
  // transition : PRE_CONTRACT_SIGNED -> DEPOSIT_PAID
}

```

```

Async completeSale(transactionId : string, notaryId : string) {
  // Vérification notariale, enregistrement
  // transition : DEPOSIT_PAID -> COMPLETED
  // Archiver tous les documents
}
}
...

```

7. PAIEMENTS MOBILE MONEY (AGRÉGATEUR CINETPAY)

Backend/src/payments/payment.service.ts

```

``typescript
Export class PaymentService {
  Async initiatePayment(dto : InitiatePaymentDto) : Promise<Payment> {
    Const payment = await this.prisma.payment.create({ data : { ... } });
    // Appel à l'API CinetPay (exemple simplifié)
    Const cinetpayResponse = await axios.post('https://api.cinetpay.com/v1/payment', {
      Apikey : process.env.CINETPAY_API_KEY,
      Site_id : process.env.CINETPAY_SITE_ID,
      Transaction_id : payment.id,
      Amount : dto.amount,
      Currency : 'XOF',
      Channels : dto.method, // 'MOBILE_MONEY'
      Customer_phone_number : dto.phone,
    });
  }
}

```

```

        Notify_url : 'https://api.aksus-immob.com/payments/callback',
    });
    // Rediriger l'utilisateur vers cinetpayResponse.data.payment_url
    Return payment ;
}

@Post('callback')
Async handleCallback(@Body() body : any) {
    // Vérifier signature, mettre à jour statut paiement
    // Déclencher l'étape suivante du workflow
}
}
'''

```

Les méthodes supportées pour le Bénin : MTN_MOMO, MOOV_MONEY, ORANGE_MONEY (via CinetPay). Aucune intégration directe complexe.

8. MULTILINGUISME (i18n côté frontend)

Fichier frontend/src/i18n/index.ts

```

'''typescript
Import i18n from 'i18next' ;
Import { initReactI18next } from 'react-i18next' ;
Import translationFR from './locales/fr.json' ;
Import translationEN from './locales/en.json' ;
Import translationPT from './locales/pt.json' ;

I18n.use(initReactI18next).init({
    Resources : {
        Fr : { translation : translationFR },

```

```
    En : { translation : translationEN },  
    Pt : { translation : translationPT },  
  },  
  Lng : 'fr',  
  fallbackLng : 'fr',  
  interpolation : { escapeValue : false },  
});
```

```
Export default i18n ;  
...
```

Exemple de clés dans fr.json :

```
```json  
{
 « welcome » : « Bienvenue sur AKSUS_Immob »,
 « search » : « Rechercher »,
 « propertyType » : « Type de bien »,
 « price » : « Prix »,
 « verifiedTitle » : « Titre vérifié »,
 « contactSeller » : « Contacter le vendeur »
}
...
```

Sélecteur de langue persistant via cookie/localStorage.

## 9. NOTIFICATIONS (EMAIL + SMS)

Backend/src/notifications/notification.service.ts

```
```typescript
```

```

Export class NotificationService {

  Async sendEmail(to : string, template : string, data : any) {

    // Utilisation de nodemailer ou SendGrid

  }

  Async sendSMS(to : string, message : string) {

    // Exemple avec Africa's Talking

    Await axios.post('https://api.africastalking.com/version1/messaging', {

      Username : process.env.AT_USERNAME,

      To,

      Message,

    }, { headers : { apiKey : process.env.AT_API_KEY } }) ;

  }

}

...

```

10. ADMINISTRATION (DASHBOARD SUPER ADMIN)

Routes réservées aux administrateurs.

```

```typescript

@Controller('admin')
@UseGuards(JwtAuthGuard, RolesGuard) // Rôle ADMIN

Export class AdminController {

 @Get('stats')

 Async getDashboardStats() {

 Return {

 totalUsers : await this.prisma.user.count(),

 totalProperties : await this.prisma.property.count(),

 totalTransactions : await this.prisma.transaction.count(),

 revenue : await this.calculateRevenue(),

 }

 }

}

```

```

};
}

@Get('users')
Async listUsers(@Query() page : number) { ... }

@Post('languages/add')
Async addLanguage(@Body() body : { code : string, name : string, translations : any }) {
 // Enregistrer les nouvelles traductions dans un fichier ou base de données
}
}
...

```

## 11. DÉPLOIEMENT SIMPLIFIÉ

- Backend : Dockerfile multi-stage (Node 18 + NestJS build, puis image légère)
- Frontend : nginx servant le build React
- CI/CD : GitHub Actions construisant les images et les poussant sur un registry (Docker Hub, GHCR)
- Production : Un serveur VPS (Ubuntu) exécutant docker-compose up -d ; pas d'orchestrateur complexe
- Sauvegardes : Script quotidien pg\_dump vers un bucket S3-compatible

Exemple de Dockerfile backend :

```

```dockerfile
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

```

```
FROM node :18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY package*.json ./
RUN npm ci --production
EXPOSE 3000
CMD ["node", "dist/main.js"]
...
```

Ce support couvre l'intégralité des sections fonctionnelles et techniques d'AKSUS_Immob avec un niveau de détail directement exploitable.

A développer avec un modèle d'interface mettant en valeur l'immobilier innovant avec la technologie et la protection de l'environnement et le cadre de vie moderne Béninois en 2026

AFRIK DATA AND TECHNOLOGY à dataafrik@gmail.com ou au +229 01 97 79 56 11.